HDOC DAY-72

NAME-PRIYADARSHNA

SAPID-590032928

BATCH-15

1. Aim

To write a C program to manage student attendance using a dynamically resizable matrix with pointers, malloc(), realloc(), and free().

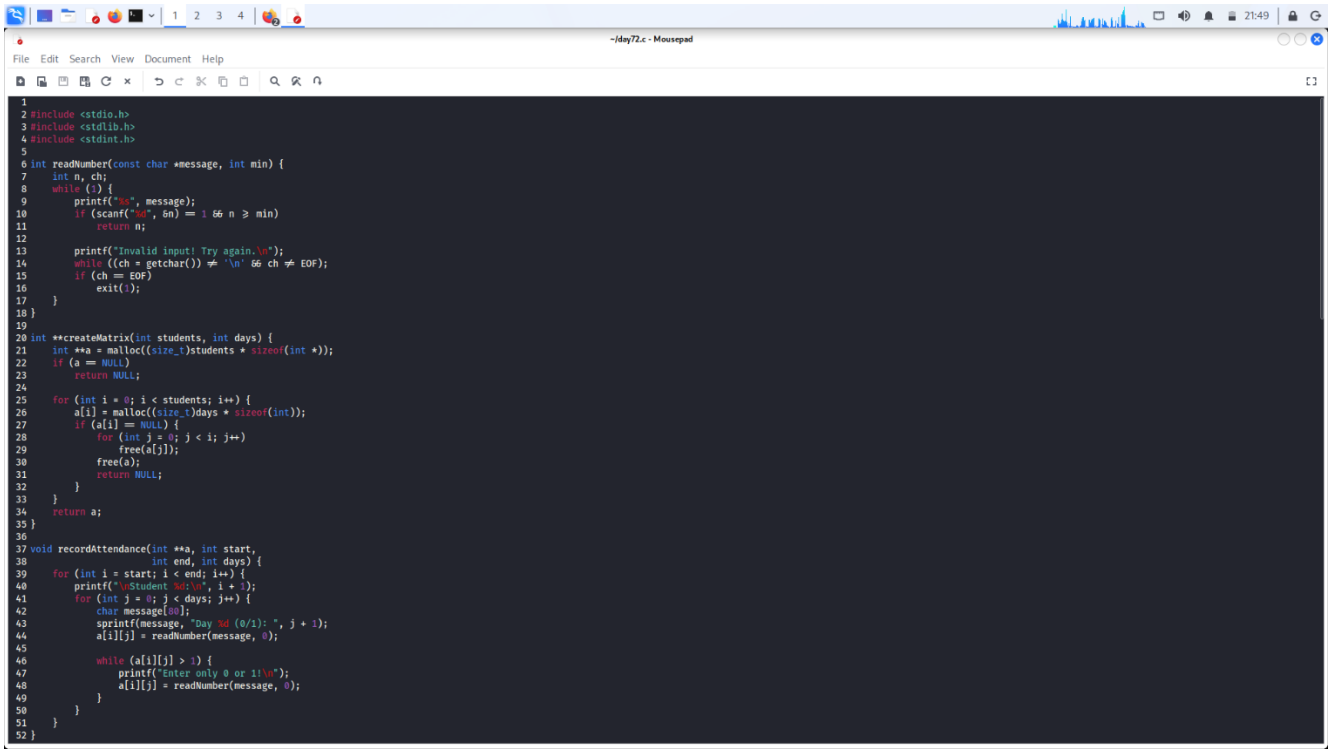
2. Problem Statement

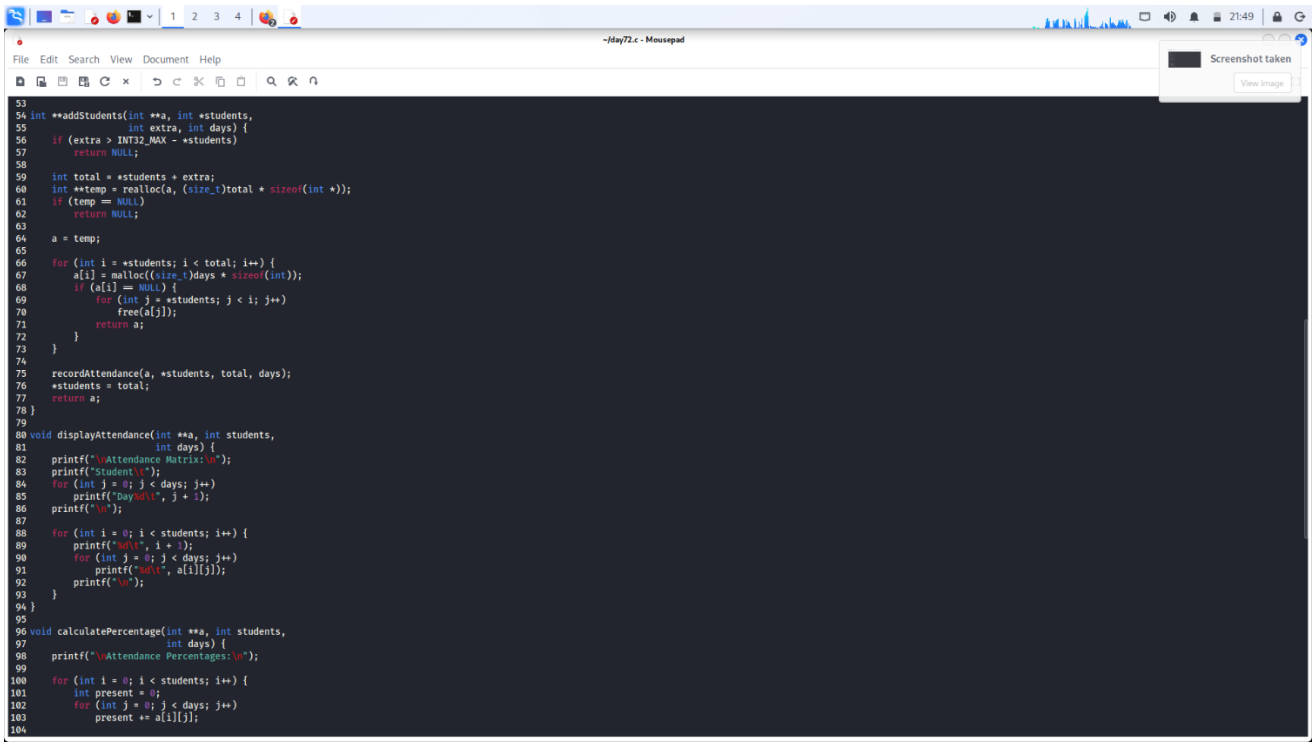
Develop a C program that stores student attendance in a dynamically allocated matrix, allows new students to be added, validates attendance inputs (0 or 1), and calculates the attendance percentage of each student.

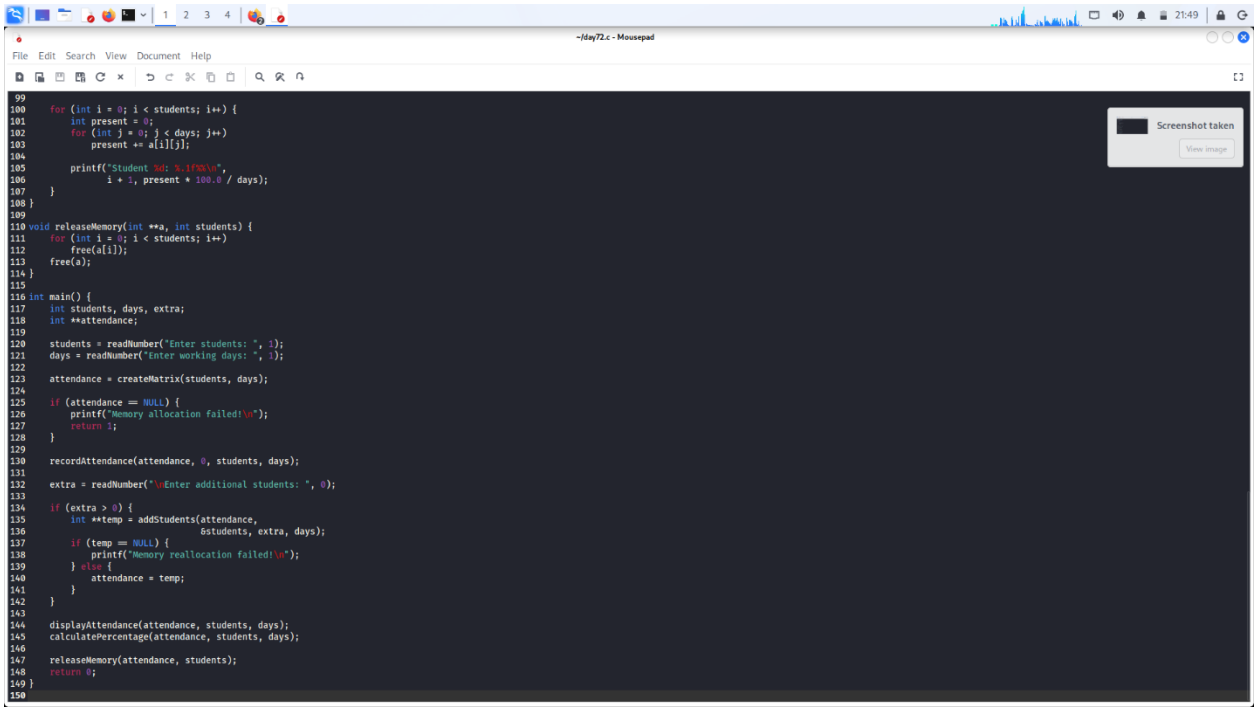
3. Algorithm

1. Start the program.
2. Read the initial number of students and working days.
3. Allocate memory dynamically using malloc().
4. Read attendance details for each student (1 = Present, 0 = Absent).
5. Read the number of additional students.
6. Resize the matrix using realloc().
7. Record attendance for newly added students.
8. Display the complete attendance matrix.
9. Calculate the attendance percentage of each student.
10. Release allocated memory using free().
11. Stop the program.

>C PROGRAM:


~/day72.c - Mousepad
File Edit Search View Document Help
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
int readNumber(const char *message, int min) {
int n, ch;
while (1) {
printf("%s", message);
if (scanf("%d", &n) == 1 && n >= min)
return n;
printf("Invalid input! Try again.\n");
while ((ch = getchar()) != '\n' && ch != EOF);
if (ch == EOF)
exit(1);
}
}
int **createMatrix(int students, int days) {
int **a = malloc((size_t)students * sizeof(int *));
if (a == NULL)
return NULL;
for (int i = 0; i < students; i++) {
a[i] = malloc((size_t)days * sizeof(int));
if (a[i] == NULL) {
for (int j = 0; j < i; j++)
free(a[j]);
free(a);
return NULL;
}
}
return a;
}
void recordAttendance(int **a, int start,
int end, int days) {
for (int i = start; i < end; i++) {
printf("\nStudent %d:\n", i + 1);
for (int j = 0; j < days; j++) {
char message[80];
sprintf(message, "Day %d (0/1): ", j + 1);
a[i][j] = readNumber(message, 0);
while (a[i][j] > 1) {
printf("Enter only 0 or 1!\n");
a[i][j] = readNumber(message, 0);
}
}
}
}


~/day72.c - Mousepad
File Edit Search View Document Help
Screenshot taken
View image
int **addStudents(int **a, int *students,
int extra, int days) {
if (extra > INT32_MAX - *students)
return NULL;
int total = *students + extra;
int **temp = realloc(a, (size_t)total * sizeof(int *));
if (temp == NULL)
return NULL;
a = temp;
for (int i = *students; i < total; i++) {
a[i] = malloc((size_t)days * sizeof(int));
if (a[i] == NULL) {
for (int j = *students; j < i; j++)
free(a[j]);
return a;
}
}
recordAttendance(a, *students, total, days);
*students = total;
return a;
}
void displayAttendance(int **a, int students,
int days) {
printf("\nAttendance Matrix:\n");
printf("Student\t");
for (int j = 0; j < days; j++)
printf("Day%d\t", j + 1);
printf("\n");
for (int i = 0; i < students; i++) {
printf("%d\t", i + 1);
for (int j = 0; j < days; j++)
printf("%d\t", a[i][j]);
printf("\n");
}
}
void calculatePercentage(int **a, int students,
int days) {
printf("\nAttendance Percentages:\n");
for (int i = 0; i < students; i++) {
int present = 0;
for (int j = 0; j < days; j++)
present += a[i][j];




```c

    for (int i = 0; i < students; i++) {
        int present = 0;
        for (int j = 0; j < days; j++)
            present += a[i][j];

        printf("Student %d: %.1f%%\n",
               i + 1, present * 100.0 / days);
    }
}

void releaseMemory(int **a, int students) {
    for (int i = 0; i < students; i++)
        free(a[i]);
    free(a);
}

int main() {
    int students, days, extra;
    int **attendance;

    students = readNumber("Enter students: ", 1);
    days = readNumber("Enter working days: ", 1);

    attendance = createMatrix(students, days);

    if (attendance == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }

    recordAttendance(attendance, 0, students, days);

    extra = readNumber("\nEnter additional students: ", 0);

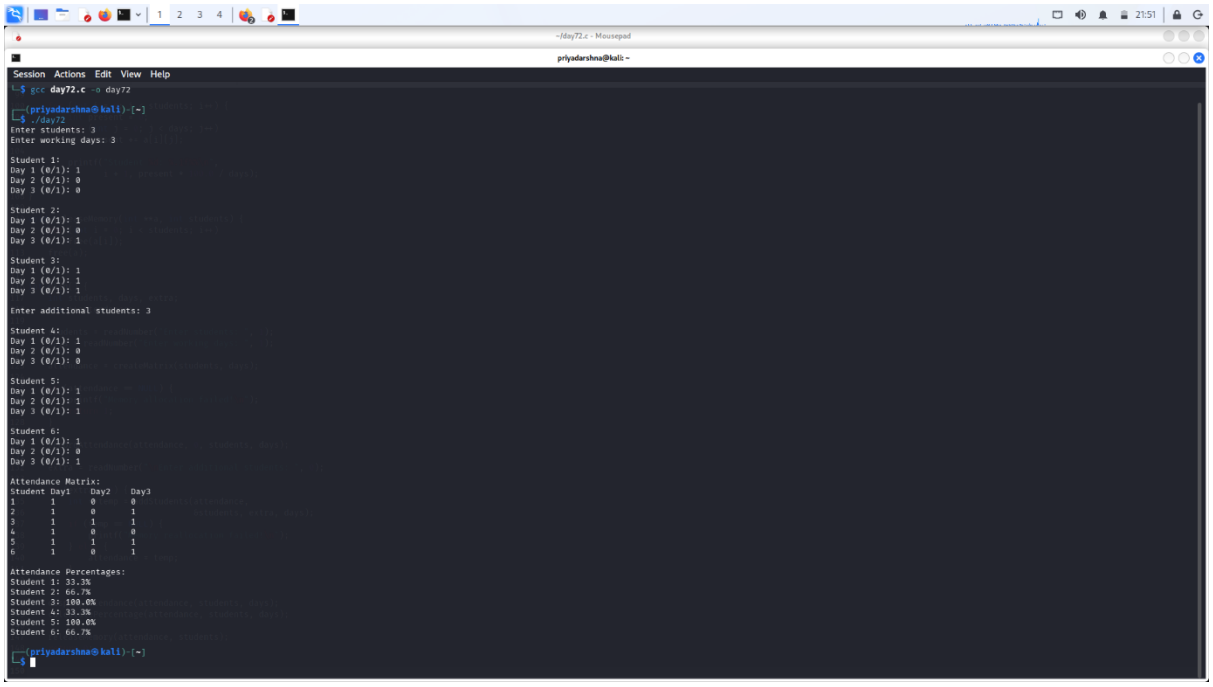
    if (extra > 0) {
        int **temp = addStudents(attendance,
                                 &students, extra, days);
        if (temp == NULL) {
            printf("Memory reallocation failed!\n");
        } else {
            attendance = temp;
        }
    }

    displayAttendance(attendance, students, days);
    calculatePercentage(attendance, students, days);

    releaseMemory(attendance, students);
    return 0;
}

```

>OUTPUT:




```
└─$ gcc day72.c -o day72

┌──(priyadarshna㉿kali)-[~]
└─$ ./day72
Enter students: 3
Enter working days: 3

Student 1:
Day 1 (0/1): 1
Day 2 (0/1): 0
Day 3 (0/1): 0

Student 2:
Day 1 (0/1): 1
Day 2 (0/1): 0
Day 3 (0/1): 1

Student 3:
Day 1 (0/1): 1
Day 2 (0/1): 1
Day 3 (0/1): 1

Enter additional students: 3

Student 4:
Day 1 (0/1): 1
Day 2 (0/1): 0
Day 3 (0/1): 0

Student 5:
Day 1 (0/1): 1
Day 2 (0/1): 1
Day 3 (0/1): 1

Student 6:
Day 1 (0/1): 1
Day 2 (0/1): 0
Day 3 (0/1): 1

Attendance Matrix:
Student Day1    Day2    Day3
1       1       0       0
2       1       0       1
3       1       1       1
4       1       0       0
5       1       1       1
6       1       0       1

Attendance Percentages:
Student 1: 33.3%
Student 2: 66.7%
Student 3: 100.0%
Student 4: 33.3%
Student 5: 100.0%
Student 6: 66.7%

┌──(priyadarshna㉿kali)-[~]
└─$
```